TRACK A: INFRASTRUCTURE-FIRST CAPSTONE

KijaniKiosk Automated Infrastructure & Multi-Stage CI/CD Pipeline

Presenter: Abraham Gitonga | Date: August 17, 2026

Zero-Downtime, Monitored Kubernetes Delivery via Terraform, Ansible & Jenkins

CONTEXT & STRATEGY

Problem Statement & Operational Gap

The Operational Bottlenecks

- **Manual Deployments:** High risk of human error, configuration drift, and lack of auditability across execution environments.
- **Single-Namespace Risk:** Shared namespace state caused resource contention and increased risk of staging changes impacting production.
- **Lack of Automated Testing:** Absence of pre-release smoke validation prior to promoting builds across environments.

The Solution Architecture

- **Declarative IaC:** Automated environment provisioning via Terraform and configuration management via Ansible.
- **Isolated Environments:** Strict multi-tenant separation using K8s namespaces (kijani-staging vs. default production).
- **Orchestrated Pipeline:** Jenkins declarative workflow with shallow Git clones, port-forward smoke testing, and approval gates.

INFRASTRUCTURE DESIGN

System Architecture & Toolchain Flow

1. Git / GitHub

SOURCE CODE

Triggers Jenkins declarative pipeline build execution.

2. Terraform

IAC PROVISIONING

Creates K8s namespaces (kijani-staging) & resource quotas.

3. Ansible

CONFIG MANAGEMENT

Generates dynamic ConfigMaps via Jinja2 templates.

4. Kubectl / K8s

WORKLOAD DELIVERY

Manages deployments, rollout status, and service routing.

5. Prometheus

CLUSTER METRICS

Scrapes pod metrics and monitors health across namespaces.

ARCHITECTURE TRADE-OFFS

Technical Decision: Decoupling Configuration & State

Strategy	Pros	Cons	Verdict
Pure Ansible (k8s module)	Single tool for configuration generation & state application.	Requires heavy Python K8s dependencies on Jenkins agent; slower status checks.	REJECTED
Decoupled Approach (Ansible + kubectl)	Lightweight agent, fast native CLI execution, reliable rollout status checks.	Requires explicit directory context management within Jenkins stages.	SELECTED

PIPELINE AUTOMATION

Pipeline Governance & Quality Gates

01

Automated Smoke Testing

Executes background port-forward (8888:8080) in staging. Runs curl validation with retries prior to release gate.

02

Interactive Approval Gate

Pauses pipeline execution post-staging. Requires explicit manual sign-off and mandatory operational rationale logging.

03

Fault Recovery & Rollback

Configured with post { failure } hooks to trigger 'kubectl rollout undo' automatically if staging fails.

GOVERNANCE & VERIFICATION

AI Tooling Governance & Manual Interventions

Domain	AI-Generated Contribution	Critical Manual Engineering Fixes
Pipeline Logic	Baseline declarative Jenkinsfile structure and stage outlines.	Fixed pipeline directory pathing (dir('kijani-kiosk-capstone')) to locate manifests correctly.
Smoke Testing	Initial curl script template.	Corrected port mapping (8888:8080) and added background process cleanup (kill $PF_PID).
Approval Gate	input parameter block design.	Standardized variable evaluation (userInput.APPROVAL_RATIONALE) for clean log output.

FUTURE IMPROVEMENTS

Production Gaps & Remediation Roadmap

Identified Production Gaps

- **Static Manifests:** Current deployment relies on separate directory manifests rather than dynamic environment overlays.
- **Basic Smoke Tests:** Validates HTTP 200 on root endpoint but does not execute synthetic transactions against database services.

Planned Remediation Roadmap

- **Kustomize Overlays:** Refactor manifests into base/ overlays for DRY environment management.
- **Chaos & Load Testing:** Integrate k6 performance tests prior to release gate approval.
- **Metric-Driven Rollbacks:** Leverage Prometheus Alertmanager to automatically trigger rollbacks on error spikes.